Agent changes — this session

Every change to the agent since the pipeline baseline (`26fa7df`, public 0.859), by author, with the before/after and the measured effect. The evaluator, catalog, public labels and API contract were **not** touched.

Score progression

checkpoint	who	public	adversarial (96)	note
pipeline baseline	team	0.859	0.684	dual-track + span rerank + FixedPolicy
+ windowed recommendations	KW	0.891	0.764	superseded by the elimination scan
+ tuned BM25 field weights	corainexia	0.898	0.725	merged onto the scan
+ elimination scan (turn 3)	KW	0.898	0.725	(this is what `main` carried)
+ facet-agreement rerank signal	corainexia	0.903	0.788	PR #3
+ retrieval-recall fixes	KW	0.903	0.788	this branch; dev 0.909, holdout 0.895
+ query facet extraction	corainexia	—	—	PR #5
+ multi-route anchor retrieval	xiaotong0329	—	—	PR #6
+ category tail match	Elinengu	**0.9128**	**0.7914**	change 5; fixes the last public miss — public 200/200
+ learned boilerplate stoplist	Elinengu	0.9128	0.7914	change 6; score-neutral by design
+ negative facet evidence	Elinengu	0.9143	0.7917	penalise contradiction of a stated facet; profile + budget signals measured and rejected — `docs/team/rerank_signals.md`
+ pair spans + word-bounded matching	Elinengu	**0.9159**	**0.7944**	keep key:value associations intact; worst hard bucket +4.7 MRR pts

Net: **public 0.859 -> 0.9159, adversarial 0.684 -> 0.7944.** 57/57 tests pass. The eight core-agent changes are detailed below; supporting tooling and docs follow.

Dashes mark changes that landed while several branches were merging in parallel, where no clean before/after was captured at the time. Changes 5-8 were measured in isolation instead — toggling the one parameter in a single process — which is the method `.claude/skills/record-change/SKILL.md` now requires, precisely because differencing across merges attributed one teammate's gain to another's change.

Change 1 — Recommendation strategy: elimination scan (Kwong Weng)

Files: `starter/agent.py`, `tools/sweep.py`

Problem

After ~turn 3 the simulated customer has disclosed all ~4 constraints and the ranking stops changing. The old agent showed the **same top 10 every turn 3-10**, so any target not in that top 10 was a guaranteed miss — 12 of 200 public sessions, and far more on generic-constraint targets.

What changed

Two iterations, both gated by `AgentConfig` flags so the old behaviour is one switch away:

1. **Windowed scan** (`b71db6d`): freeze the reranked pool on the first list shown, then walk it in windows — turn 3 ranks 1-10, turn 4 ranks 11-20, turn 5 ranks 21-30, … Re-snapshot on a real new constraint or an intent override. `AgentConfig.scan_windows`.
2. **Elimination scan** (`96d5385`, ships): a product shown and not hit on is a *confirmed non-target* (the session would have ended). So each turn: drop everything already shown, return the top 10 of the re-ranked **survivors**. No frozen pool, no cursor — `rerank()` runs every turn anyway, so new constraints and overrides are reflected automatically, and a target that drifts down the ranking is still a survivor and surfaces a turn later (no gap).

```
# starter/agent.py _shortlist(), simplified
shown = self._shown.setdefault(sid, set())
if (state.override_turn or 0) != self._shown_override.get(sid, 0):
 shown.clear() # a pre-override list confirms nothing
 self._shown_override[sid] = state.override_turn or 0
picks = [asin for asin, _ in candidates if asin not in shown][:limit]
shown.update(picks)
```

New `AgentConfig` fields: `elimination_scan = True`, `hold_until_stalled = False`. `first_recommend_turn` stays the start-turn knob; a dev+holdout sweep (`tools/sweep.py`: `elim1/2/3`, `elim_hold`) picked **turn 3** — earlier starts freeze a poor MRR on under-informed early hits.

One correctness fix

An intent-override session's pre-override list often already contains the target, but the evaluator ignores hits until the override lands. Without clearing the shown-set on override detection the scan sails past it — this collapsed override to 0.11 in one iteration; the clear restores it to 1.00.

Effect

Public 0.891 (windowing) then elimination same on public but **fixes the boundary-MRR regression** (0.756 -> 0.883) and is simpler. Public hit@10 0.940 -> 0.990; only 2 misses left. On the adversarial set the two variants trade (windowing 0.764, elimination 0.725) — elimination re-ranks every turn, so on all-generic constraints the "no preference" junk spans jostle a borderline target away. Change 4 fixes that.

Change 2 — Tuned BM25 field weights (corainexia, PR #2)

File: `src/index.py` commit `201c00d`

```
DEFAULT_WEIGHTS (parent_asin, title, categories, features, details, store, description)
 before (0.0, 6.0, 4.0, 2.5, 2.5, 1.5, 1.0)
 after (0.0, 8.0, 5.0, 6.0, 6.0, 0.5, 0.25)
```

Heavier title / categories / features / details; store and long marketing description almost silenced. The original tuple was inherited from the starter and had never been tuned.

Effect

Public 0.8928 -> **0.8982** (+0.005), almost entirely MRR (0.804 -> 0.821) — hit@10 was already saturated, so this ranks the found target higher. Every scenario's MRR improved; holdout rose more than dev, so it generalises.

Change 3 — Facet-agreement rerank signal (corainexia, PR #3)

File: `src/rerank.py` commit `2d7f92b`

Adds a fourth term to the rerank score. `_facet_agreement()` runs `src/facets.py:extract()` over the customer's accumulated text and over each candidate, and counts matching facet values (material, colour, size, style, use_case):

```
total = ( span_weight * coverage
 + retrieval_weight * (bm25 / max_bm25)
 + popularity_weight * popularity
 + facet_weight * facet_score ) # facet_weight = 0.3 <-- new
```

This is `docs/team/ideas.md` Idea 2 (partial) — the reranker had been ignoring `FacetStore`, which is built for every product but read by nothing.

Effect

Public 0.8995 -> **0.9031**; recovers the MRR that Change 4's query filter costs on its own and adds more (public MRR 0.814 -> 0.825).

Change 4 — Retrieval recall: hold declines out of the query (Kwong Weng)

Files: `src/state.py`, `src/rerank.py` commit `960eed0`

Problem

`retrieve()` queries `state.full_text()` = every utterance joined. From turn 4 the simulator's *"I don't have an additional preference for feature / use_case / style / material / colour / size"* leaks `feature, style, material, colour, size, category` into the BM25 OR-query and the span matcher — terms that match huge swathes of the catalog and dilute the target. Recall **degrades within a session**: adversarial "target in the 300-pool" fell from 96% (turn 3) to 81% (turn 10).

What changed

- `Utterance.declined` flag, set in `observe()` when `NO_PREFERENCE_CUES` matches. `full_text()`, `focused_text()` and `query_spans()` skip declined utterances; a decline also no longer counts as a "productive" turn.
- `RerankConfig.depth 200 -> 300` — rescore the whole retrieval pool, not a prefix. ~12% of cluster-target sessions had the target in the pool but past rank 200, where the span signal never applied. (No-op on public; helps only the adversarial set.)

Effect — recall (the point of the change)

target in 300-pool at end of session	before	after
public	98%	**100%**
adversarial	81%	**95%**
adversarial: degenerate_card / homogeneous_cluster	56% / 81%	**88% / 100%**
end-of-session pool-rank p90 (public / adversarial)	87 / 279	**16 / 200**

The within-session degradation is gone. Score: adversarial 0.725 -> **0.788** (hit@10 0.802 -> 0.885); public flat on its own, net **+0.005** once combined with Change 3.

Change 5 — Category tail match in the reranker (Elinengu)

Files: `src/rerank.py` — commit `f322a52`. Full write-up: `docs/team/category_tail_match.md`

Problem

The customer's opening names the target's coarse category, which the evaluator builds from the two most specific levels of the target's category path (`coarse_category`, `evaluator/local_evaluator.py:126`): `Novelty > Women` becomes *"I'm looking for Novelty Women"*.

Ancestor overlap alone cannot exploit that. A deeper candidate — `... > Novelty > Women > Tops & Tees > T-Shirts` — shares every ancestor the target has and scores just as well, even though its own two most specific levels (`Tops & Tees T-Shirts`) are never mentioned in the opening. In the last remaining public-set miss (`public_0020`) this produced a **159-way tie** in the reranker.

What changed

Score the *tail* of each candidate's category path, not its ancestors: a point for each of the candidate's two most specific levels whose tokens are fully contained in the opening. Matching is by token containment rather than by parsing the opening's template, so paraphrased private-set wording still works. Generic levels such as `Clothing, Shoes & Jewelry` are excluded.

```
for part in cleaned[-2:]:                    # the two most specific levels
    part_tokens = set(terms(part))
    if part_tokens and part_tokens <= opening_terms:
        score += 1.0
```

`RerankConfig.tail_weight = 0.8`. The 0.6-1.5 range scores identically on dev and holdout, so this sits mid-plateau rather than at either split's argmax.

Effect

Measured in one process by toggling `tail_weight` between `0.0` and `0.8`, everything else held constant.

	off	on	delta
Public set	0.907000	**0.912801**	+0.0058
Public MRR	0.8273	0.8417	+0.0144
Public MTTC	3.060	2.985	-0.075
Adversarial set	0.786037	**0.791375**	+0.0053
Adversarial MRR	0.6798	0.6949	+0.0151

The gain is entirely in ranking. Hit rate does not move on either set (public 1.000, adversarial 0.8854) — the tail match reorders candidates retrieval had already found, which is what a reranking signal should do.

Change 6 — Learned boilerplate stoplist (Elinengu)

Files: `tools/build_stoplist.py`, `src/stoplist.py`, `src/text.py`, `tests/test_stoplist.py` — commit `f322a52`

Problem

`BOILERPLATE` in `src/text.py` was 24 hand-written tokens with no record of what evidence produced any of them. `IMPLEMENTATION.md` proposed replacing it by dropping the most frequent ~200 catalog terms.

That proposal was wrong. Ranked by document frequency over the 50,000-product catalog, `polyester` is 72nd, `cotton` 83rd, `black` 97th, `leather` 111th and `spandex` 141st. Dropping the top 200 deletes every one — and those are precisely the constraints the customer discloses, because `intent_card()` inserts a material at position 0 and a colour at position 1 of every card. Meanwhile `asin`, a genuine member of the hand-written list, sits at rank 10,379 with 0.0% document frequency. Frequency fails in both directions.

What changed

The usable signal is *where* a token occurs, not how often. Structural metadata lives in the `details` dict and appears almost nowhere else:

structural	in `details`	attribute	in `details`
`department`	100.0%	`spandex`	1.2%
`dimensions`	99.6%	`cotton`	2.5%
`manufacturer`	99.6%	`polyester`	3.3%
`inches`	97.7%	`black`	13.4%

The gap between 16% and 96% is empty of attribute words, so the threshold is read off the catalog's own distribution rather than tuned against a score. `tools/build_stoplist.py` selects tokens with document frequency >= 5% and >= 90% of occurrences inside `details`, and writes `src/stoplist.py`. It reads `data/catalog.jsonl` and never opens `data/public_set.jsonl`, so it cannot fit the public sessions.

The learned list reproduces 14 of the 24 hand-written terms and finds 22 more nobody wrote down — every month name and year 2014-2022, harvested from `Date First Available: August 15, 2019`.

Ten terms stay hand-written, deliberately. `imported`, `machine`, `wash`, `closure`, `care` and friends are care-and-origin phrases in `features/description`. Two statistics were tested and both fail to separate them from real attributes: by document frequency `closure` (38.6%) outranks `polyester` (21.8%); by spread across the 12 largest category buckets `polyester` (CV 0.51) sits between `imported` (0.49) and `wash` (0.60), with `black`/`white` (0.34/0.31) inside the scaffolding band. What separates them is that a shopper does not choose between "imported" and "not imported" — semantics, not frequency. The evidence sits beside them in `src/text.py` so the experiment is not retried.

Effect

Measured in one process by swapping `src.text.BOILERPLATE` between the two lists.

	hand-written (24)	learned (46)
Public set	0.912801	**0.912801**
Adversarial set	0.791636	0.791375

	hand-written (24)	learned (46)
Sweep dev / holdout	0.9187 / 0.9029	0.9190 / 0.9029

Score-neutral, as predicted before it was written. Boilerplate removal strips a median of one token from a ten-token query, and `MAX_QUERY_TERMS = 60` never binds — 0 of 200 sessions come close. The adversarial `-0.0003` is one session's reciprocal rank; hit rate and MTTC are unchanged on both sets.

It ships for auditability and for robustness on a catalog nobody has hand-inspected, not for points. `tests/test_stoplist.py` holds the invariants that keep a future regeneration honest.

Change 7 — Negative facet evidence in the reranker (Elinengu)

Files: `src/rerank.py`, `src/facets.py`, `src/policy.py`, `tools/sweep.py`, `tests/test_components.py` — full record with the two rejected sibling signals in `docs/team/rerank_signals.md`

Problem

Every rerank term is positive evidence: span coverage, facet agreement, category and tail alignment. None can separate a candidate that *satisfies* "color: grey" from one that merely *doesn't contradict* it — a black-only shirt matches "cotton shirt" spans exactly as well as a grey one and loses nothing for being black. The customer's constraint rules products out; the scorer only ever ruled products in.

What changed

`_facet_conflicts()` counts stated facet values the candidate contradicts, and the score subtracts `facet_conflict_weight (0.4) * conflicts`. A conflict requires all three of:

1. the customer stated a value for the attribute (`extract_query_facets`);
2. the candidate resolves that attribute too — **silence is never punished**, missing data is not disagreement;
3. the stated value (and every alias — `grey`/`gray` is the vocabulary's only synonym pair) appears nowhere in the candidate's text as a word-bounded substring. This guards against `extract()`'s first-match-wins: a "black/grey reversible" belt extracts `color: black` but still contains "grey" and must not be penalised.

The override fix the first version needed. Judged against `full_text()`, the penalty punished the target for *obeying an intent override*: after "actually, ignore black — I need grey", the stale "black" was still extracted first and grey-only products were demoted for contradicting it — the adversarial `generic_override` bucket dropped 0.673 -> 0.626 MRR. Conflicts are therefore judged against `extract_query_facets(state.focused_text())` — the currently authoritative turns, identical to `full_text()` until an override fires — which restored the bucket to exactly 0.673. Positive terms still read `full_text()` deliberately: stale positive evidence mildly boosts wrong candidates, stale negative evidence actively demotes the right one.

Weight 0.4 sits at the low end of the dev plateau (0.9224 at 0.4, 0.9226 at 0.8): a penalty term gets the smallest weight that works, and at 0.4 one conflict can reorder saturated ties but cannot overturn a genuine span lead (each matched span is worth >= 1.12).

Effect

Measured in one process by toggling `facet_conflict_weight` between 0.0 and 0.4:

	off	on
Public set	0.912801	**0.914287** (hit stays 200/200)
Sweep dev / holdout	0.9207 / 0.9010	0.9224 / 0.9021
Adversarial set	0.791375	0.791697 — no bucket regresses

The signal was designed for the `homogeneous_cluster` bucket, and that hypothesis was wrong: bucketmates share the stated facets by construction, so there is nothing to contradict inside the cluster (+0.002 only). The gain is cross-cluster precision on the general distribution. Five unit tests hold the guards (multi-value products, silence, override staleness, weight-0 no-op).

Two sibling signals were measured and **not** shipped, per the remove-dead-options convention: profile-weighted facet agreement (dev +0.0013 does not reproduce on holdout, -0.0008 — the customer already discloses their profiled preferences verbatim, so the profile is redundant with the transcript) and budget/price closeness (rejected before implementation: 0 of 200 public intent cards contain a budget, 0.45% of the catalog can produce one). `docs/team/rerank_signals.md` carries both measurements.

Change 8 — Pair spans and word-bounded span matching (Elinengu)

Files: `src/text.py`, `src/state.py`, `src/rerank.py`, `tools/sweep.py`, `tests/test_components.py` — full record in `docs/team/rerank_signals.md` §2

Problem

Spotted by reading the `public_0020` transcript. The customer's disclosure "Heather Grey: 90% Cotton, 10% Polyester" is a mapping — colour-variant → composition — but `constraint_spans()` splits on `[.;:,\n]`, severing "heather grey" from *its* "90 cotton 10 polyester". A candidate that pairs the composition differently (an 80/20 heather grey) matches all the fragments exactly as well as the target. Fragments ask "mentions 90% cotton at all?"; the message's evidence is "says it *about heather grey*?"

Second, latent bug: coverage tested `span in text` unanchored, so "90 cotton" also matched "190 cotton" (1-4 catalog products per numeric span).

What changed

- `pair_spans()` (`src/text.py`): splits only on sentence separators (`.;\n`, `" - "`), keeping colon/comma-joined key:value content together; strips the leading simulator framing; minimum 3 words. Catalog copy repeats these blocks verbatim, so the joined form is still an exact substring of the target: df("heather grey 90 cotton 10 polyester") = 511 products vs 612 for "heather grey" alone.
- `query_pair_spans()` (`src/state.py`): same turn-1/declined exclusions as `query_spans()`, and drops anything already emitted as a fragment so no evidence is counted twice.
- `rerank()` adds `pair_weight (0.8) x matched_pairs`, flat 1.0 per pair — the pair's value is the intact association, not its length. Both fragment and pair matching are now word-bounded (the product text is token-joined, so padding with single spaces anchors spans at token edges).

What was tried first and rejected: the obvious diagnosis — eight fragments from one line over-weight that line — led to three de-weighting prototypes: per-utterance groups as sum/√k (public 0.9139), mean (0.9134), best-single-span (0.9025). All flat or worse; the 8-of-8-vs-5-of-8 fragment gradient is load-bearing. The fix is adding the lost association, not weakening the fragments.

Effect

Measured in one process, one toggle at a time:

	baseline	+ word-bounded	+ pair spans (0.8)
Public set	0.914287	0.914937	**0.915887** (hit stays 200/200)
Adversarial set	0.791697	0.791697	**0.794375**
Sweep dev / holdout	0.9224 / 0.9021	0.9223 / 0.9040	0.9233 / 0.9048

The hard-set gain sits entirely in `homogeneous_cluster` (MRR 0.431 -> **0.478**) and `generic_override` (0.673 -> 0.680); no other bucket moves, no hit is lost. This is the bucket Change 7 could not touch: bucketmates share every fragment by construction, but they do not pair the compositions the same way — the intact association is the discriminator that survives saturation. `public_0020` itself goes from rank 4 to rank 1. Weight 0.4-1.5 scores identically on dev and holdout; 0.8 sits mid-plateau. Six new unit tests.

Supporting work (Kwong Weng)

file	what
`tools/hard_cases.py`	Adversarial session generator + per-bucket scorer. Scans the frozen catalog, buckets every product by an adversarial property, samples 16 each. `--run` scores the agent grouped by bucket.
`data/hard_set.jsonl`	96 generated sessions (6 buckets: homogeneous_cluster, budget_only_signal, boilerplate_soft, degenerate_card, generic_override, cross_category_collision). Public-set schema; scored by the unmodified evaluator.
`tools/sweep.py`	`build_configs()` — added `plain`, `elim1/2/3`, `elim_hold1/2` for the start-turn sweep.
`docs/team/ideas.md` / `ideas.pdf`	Reranking & recommendation-strategy ideas, each with the measured result: elimination scan (1a/1b), decline filter (1c), facet / category / MMR / learned-weights (2-6).
`docs/team/hard_cases.md` / `.pdf`	Failure analysis of the adversarial set and the prioritised fix plan.
`agent_summary.pdf`	Rewritten (`c7757af`) for the current elimination-scan workflow: the loop, one turn stage-by-stage, recommendation timing, and a round-by-round table per scenario.
`examples.pdf`	Two annotated agent<->simulator transcripts (one hit, one miss).

Also merged this session (teammates, non-agent)

commit	author	what
`a13d8b5`	Elinengu	`Customer_Simulator_Explainer.pdf`
`80bd4f8`	Elinengu	`tools/observe.py`, `tests/test_observe.py` — session observer / monitor
`0f085ec`	Elinengu	`docs/team/intent_override_retrieval.{md,pd`
`01ffdcb`	Elinengu	`docs/team/reranking_explained.{md,pdf}`

commit	author	what
(this change)	Elinengu	`docs/team/category_tail_match.{md,pdf}` — the last public-set miss, and the no-overfit evidence

Current agent settings (after all changes)

stage	knob	value	file
S1 index	FTS5 field weights (title, cat, features, details, store, desc)	8, 5, 6, 6, 0.5, 0.25	`src/index.py`
S5 retrieval	pool size / RRF k / focused-route weight	300 / 60 / 0.8	`src/retrieval.py`
S6 rerank	span / retrieval / popularity / **facet** weight	1.0 / 1.0 / 0.02 / **0.3**	`src/rerank.py`
S6 rerank	category (ancestor) / **tail** weight	0.4 / **0.8**	`src/rerank.py`
S6 rerank	**facet-conflict penalty** (vs post-override facets)	**0.4**	`src/rerank.py`
S6 rerank	**pair-span weight** (key:value associations, word-bounded)	**0.8**	`src/rerank.py`
S6 rerank	length bonus / **depth**	0.12 / **300**	`src/rerank.py`
S3 state	pre-override utterance weight	0.35 (largely inert)	`src/state.py`
S3 state	**declined utterances held out of every retrieval view**	—	`src/state.py`
S4 policy	FixedPolicy: `other`, then feature-ladder	—	`src/policy.py`
S7 timing	first_recommend_turn / confidence margin / earliest	3 / 0.20 / 2	`starter/agent.py`
S7 timing	**elimination_scan / hold_until_stalled**	on / off	`starter/agent.py`

Not touched (organizer-owned)

`evaluator/local_evaluator.py`, `data/catalog.jsonl`, `data/public_set.jsonl`, and the five frozen files at the root of `docs/`: `agent_api_contract.json`, `baseline_results.json`, `competition_specification.md`, `evaluation_config.json`, `submission_rules.md`.

Everything under `docs/team/` is ours — see `docs/README.md` for the split.